

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М. А. БОНЧ-БРУЕВИЧА»
(СПбГУТ)

ОТЧЕТ
ЛАБОРАТОРНАЯ РАБОТА №4

Мониторинг локальной сети Ubuntu/Windows

Руководитель

подпись, дата

Панков А. В.

Исполнитель, группа
ИКПИ-32

подпись, дата

Филимонов Д.Е.

Санкт-Петербург 2025

Цели:

- Изучить утилиты: - Ping.
- Traceroute.
- Wireshark.
- Изучить результаты команд Ping, traceroute.
- Понять результат обращения к сайту (протокол http).

Технические требования:

- Компьютер с установленной ОС Ubuntu или компьютер с установленной ОС Windows, а также ПО Wireshark.

Задачи: 1. Исследовать работу утилиты

ping:

- Проверить доступность узла (например, google.com) с разным количеством пакетов (-n в Windows, -c в Linux).
- Отправить пакеты увеличенного размера (-l в Windows, -s в Linux), сравнить результаты (RTT, потери).

2. Анализировать маршрутизацию через tracert/traceroute:

- Определить цепочку маршрутизаторов до выбранного домена (например, yandex.ru).
- Выявить узлы с высоким временем отклика или потерей пакетов, предположить причины.

3. Изучить HTTP-трафик в Wireshark:

- Зафиксировать процесс установки TCP-соединения (SYN, SYNACK, ACK).
- Найти HTTP GET-запрос и ответ сервера, проверить открытость данных (логины, cookies).

4. Сравнить ICMP и HTTP:

- Проанализировать структуру пакетов (через фильтры Wireshark), выделить различия в назначении и содержимом.

5. Диагностировать сетевые проблемы:

- Сымитировать сбой (обрыв связи, перегрузку), выполнить ping и tracert, интерпретировать изменения в статистике.

Ход работы:

Часть 1

```
C:\Users\PC>ping 8.8.8.8

Обмен пакетами с 8.8.8.8 по с 32 байтами данных:
Ответ от 8.8.8.8: число байт=32 время=7мс TTL=106
Ответ от 8.8.8.8: число байт=32 время=7мс TTL=106
Ответ от 8.8.8.8: число байт=32 время=6мс TTL=106
Ответ от 8.8.8.8: число байт=32 время=7мс TTL=106

Статистика Ping для 8.8.8.8:
    Пакетов: отправлено = 4, получено = 4, потеряно = 0
    (0% потерь)
Приблизительное время приема-передачи в мс:
    Минимальное = 6мсек, Максимальное = 7 мсек, Среднее = 6 мсек
```

Рис. 1 – Проверка доступности сервера Google (8.8.8.8)

```
C:\Users\PC>ping -n 10 google.com

Обмен пакетами с google.com [64.233.162.139] с 32 байтами данных:
Ответ от 64.233.162.139: число байт=32 время=15мс TTL=108
Ответ от 64.233.162.139: число байт=32 время=15мс TTL=108
Ответ от 64.233.162.139: число байт=32 время=15мс TTL=108
Ответ от 64.233.162.139: число байт=32 время=15мс TTL=108
Ответ от 64.233.162.139: число байт=32 время=15мс TTL=108
Ответ от 64.233.162.139: число байт=32 время=15мс TTL=108
Ответ от 64.233.162.139: число байт=32 время=15мс TTL=108
Ответ от 64.233.162.139: число байт=32 время=15мс TTL=108
Ответ от 64.233.162.139: число байт=32 время=15мс TTL=108
Ответ от 64.233.162.139: число байт=32 время=15мс TTL=108

Статистика Ping для 64.233.162.139:
    Пакетов: отправлено = 10, получено = 10, потеряно = 0
    (0% потерь)
Приблизительное время приема-передачи в мс:
    Минимальное = 15мсек, Максимальное = 15 мсек, Среднее = 15 мсек
```

Рис. 2 – Проверка доступности доменного имени с ключом «-n»

```
C:\Users\PC>ping -l 150 google.com

Обмен пакетами с google.com [64.233.162.139] с 150 байтами данных:
Ответ от 64.233.162.139: число байт=150 время=15мс TTL=107
Ответ от 64.233.162.139: число байт=150 время=15мс TTL=107
Ответ от 64.233.162.139: число байт=150 время=15мс TTL=107
Ответ от 64.233.162.139: число байт=150 время=15мс TTL=107
т проследить маршрут следования данных до
Статистика Ping для 64.233.162.139:
    Пакетов: отправлено = 4, получено = 4, потеряно = 0
    (0% потерь)
Приблизительное время приема-передачи в мс:
    Минимальное = 15мсек, Максимальное = 15 мсек, Среднее = 15 мсек
```

Рис. 2 – Проверка доступности доменного имени с ключом «-l»

Промежуточный итог:

Команда `ping` позволяет диагностировать доступность узла в сети и измерять параметры соединения. Использование ключей `-n` и `-l` расширяет её функционал:

1. `-n` (количество запросов)

> По умолчанию `ping` отправляет 4 пакета (Windows) или работает до прерывания (Linux).

> Ключ `-n <число>` задаёт точное количество эхо-запросов. Например, `ping -n 10 example.com` отправит 10 пакетов.

- Польза:

> Позволяет собрать статистику по потерям пакетов и стабильности RTT (Round Trip Time) за определённый период.

> Удобен для автоматизированного тестирования (например, в скриптах).

2. `-l` (размер пакета)

> По умолчанию размер пакета — 32 байта (Windows) или 56 байт (Linux).

> Ключ `-l <размер>` задаёт размер отправляемого буфера. Например, `ping -l 1024 example.com` отправит пакеты размером 1024 байта.

- Польза:

> Позволяет проверить, как сеть справляется с передачей крупных пакетов (например, выявить фрагментацию или потери при превышении MTU).

> Увеличение размера может помочь обнаружить проблемы с пропускной способностью или загруженностью канала.

```

C:\Users\PC>tracert google.com

Трассировка маршрута к google.com [64.233.162.139]
с максимальным числом прыжков 30:
Лабораторная работа 7.2 К.

  1  <1 ms    <1 ms    <1 ms    192.168.1.1
  2  29 ms    38 ms    21 ms    5x19x0x184.static-business.spb.ertelecom.ru [5.19.0.184]
  3   3 ms     2 ms     2 ms    5x19x0x225.static-business.spb.ertelecom.ru [5.19.0.225]
  4  12 ms    12 ms    10 ms    gw-as9049.retn.net [87.245.255.183]
  5  10 ms    10 ms    10 ms    139.45.224.66
  6  14 ms    18 ms    17 ms    ae0-4.RT.DL.MSK.retn.ru [139.45.243.5]
  7  14 ms    13 ms    12 ms    72.14.222.22
  8  *        *        *        Превышен интервал ожидания для запроса.
  9  27 ms    17 ms    21 ms    192.178.241.234
 10  17 ms    16 ms    16 ms    142.251.49.24
 11  16 ms    15 ms    15 ms    72.14.238.168
 12  15 ms    15 ms    16 ms    142.250.56.127
 13  *        *        *        Превышен интервал ожидания для запроса.
 14  *        *        *        Превышен интервал ожидания для запроса.
 15  *        *        *        Превышен интервал ожидания для запроса.
 16  *        *        *        Превышен интервал ожидания для запроса.
 17  *        *        *        Превышен интервал ожидания для запроса.
 18  *        *        *        Превышен интервал ожидания для запроса.
 19  *        *        *        Превышен интервал ожидания для запроса.
 20  *        *        *        Превышен интервал ожидания для запроса.
 21  *        *        *        Превышен интервал ожидания для запроса.
 22  16 ms    15 ms    15 ms    li-in-f139.1e100.net [64.233.162.139]

Трассировка завершена.
C:\Users\PC>

```

Рис. 4 – Изучение маршрута до адреса google.com с помощью команды «tracert»

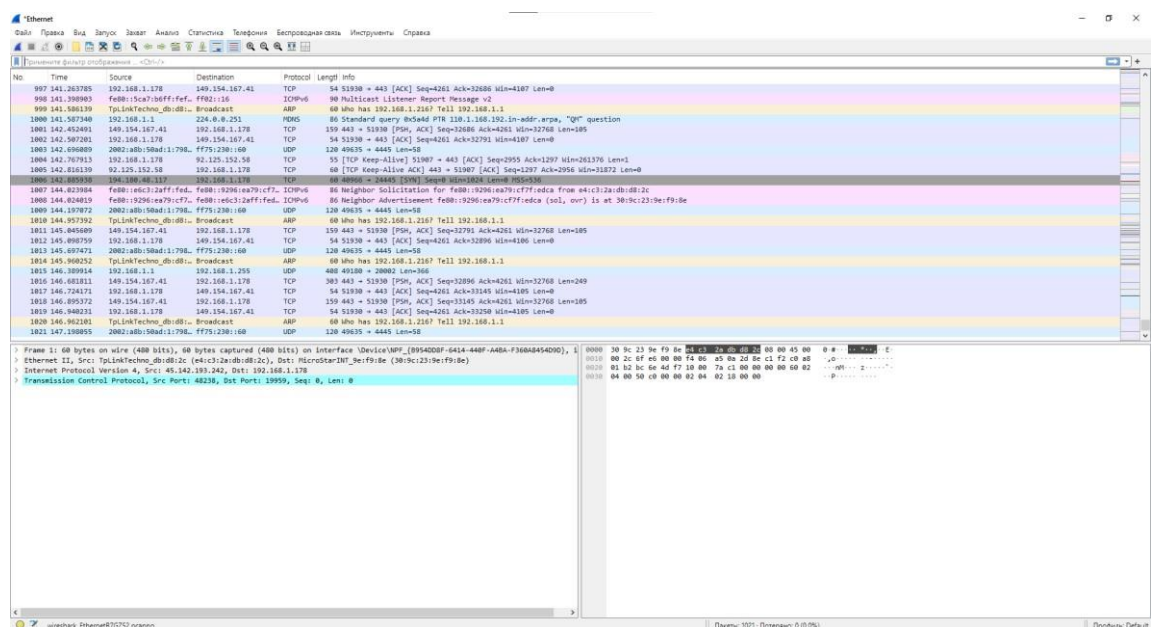


Рис. 5 – Вывод пакетов, а также их расшифровка с помощью программы «Wireshark»

Промежуточный итог по результатам работы с командами tracert и Wireshark:

1. Изучение маршрута до адреса google.com с помощью tracert

- > Команда tracert (Windows) определила цепочку маршрутизаторов (хопов), через которые проходят пакеты до целевого узла google.com. ◦

Ключевые наблюдения:

- > Общее количество хопов отражает сложность маршрута. Например, 10–15 переходов типичны для глобальных запросов.
- > Узлы с высоким временем отклика (например, 150+ мс) или отсутствием ответа (*) могут указывать на:
 - > Перегрузку канала.
 - > Фильтрацию ICMP-пакетов на промежуточных узлах.
 - > Географическую удалённость серверов.
- > Разные IP-адреса в цепочке демонстрируют динамическую маршрутизацию в глобальной сети.

2. Анализ пакетов в Wireshark

- > Wireshark зафиксировал процесс обращения к сайту через HTTP, включая:
 - > **TCP-рукопожатие** (SYN → SYN-ACK → ACK) для установки соединения.
 - > **HTTP GET-запрос** с метаданными (например, User-Agent, Host).
 - > **HTTP-ответ** сервера (например, 200 OK с передачей данных HTML/CSS).

◦ Ключевые наблюдения:

- > Отсутствие шифрования в HTTP: содержимое пакетов (логины, cookie) передаётся в открытом виде.
- > Заголовки пакетов содержат информацию об IP-адресах клиента и сервера, портах и флагах протокола.

> DNS-запросы, предшествующие HTTP-соединению, подтверждают роль системы доменных имён.

✦ **Итог:** Комбинация `tracert` и Wireshark даёт полную картину работы сети — от маршрутизации до анализа содержимого трафика, что критически важно для диагностики и оптимизации сетевых взаимодействий.

Часть 2

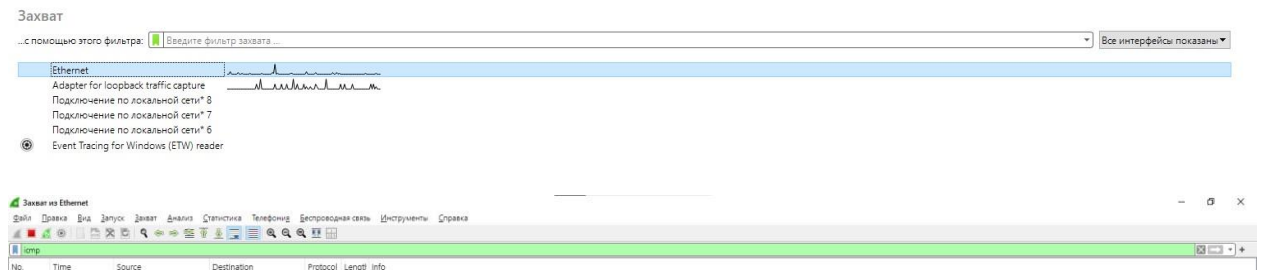


Рис. 6 – Выбор основного сетевого интерфейса

```
C:\Users\PC>ping -n 5 yandex.ru

Обмен пакетами с yandex.ru [77.88.55.88] с 32 байтами данных:
Ответ от 77.88.55.88: число байт=32 время=17мс TTL=51
Ответ от 77.88.55.88: число байт=32 время=17мс TTL=51
Ответ от 77.88.55.88: число байт=32 время=17мс TTL=51
Ответ от 77.88.55.88: число байт=32 время=17мс TTL=51
Ответ от 77.88.55.88: число байт=32 время=17мс TTL=51

Статистика Ping для 77.88.55.88:
    Пакетов: отправлено = 5, получено = 5, потеряно = 0
    (0% потерь)
Приблизительное время приема-передачи в мс:
    Минимальное = 17мсек, Максимальное = 17 мсек, Среднее = 17 мсек
```

Рис. 7 – Проверка связи с сервером Yandex.ru

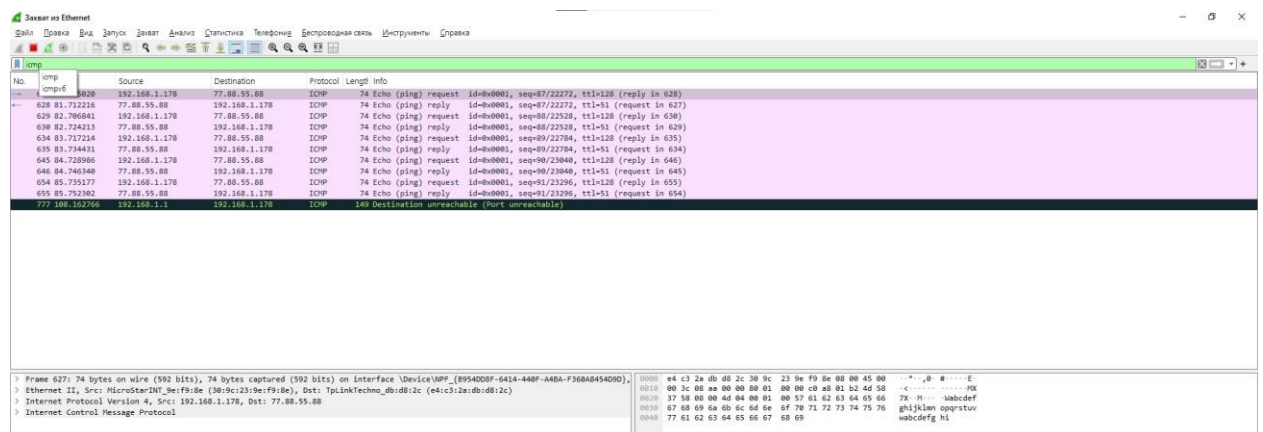


Рис. 8 – Результат с фильтрацией по протоколу «icmp»

```
C:\Users\PC>tracert yandex.ru
Трассировка маршрута к yandex.ru [77.88.55.88]
с максимальным числом прыжков 30:
  0  1  2  3  4  5  6  7  8  9
  <1 ms  <1 ms  <1 ms  192.168.1.1
  12 ms  6 ms  7 ms  5x19x0x194.static-business.spb.ertelecom.ru [5.19.0.194]
  4 ms  2 ms  2 ms  5x19x0x197.static-business.spb.ertelecom.ru [5.19.0.197]
  3 ms  3 ms  3 ms  bbr01.spb.ertelecom.ru [188.234.129.214]
  2 ms  2 ms  2 ms  31x131x196x151.static.ertelecom.ru [31.131.196.151]
  17 ms  17 ms  17 ms  sas-32z8-lag-3.yndx.net [87.250.239.25]
  19 ms  18 ms  18 ms  10.32.83.1
  20 ms  20 ms  23 ms  10.1.3.1
  17 ms  17 ms  17 ms  yandex.ru [77.88.55.88]

Трассировка завершена.
```

Рис. 9 – traceroute до сервера Yandex.ru

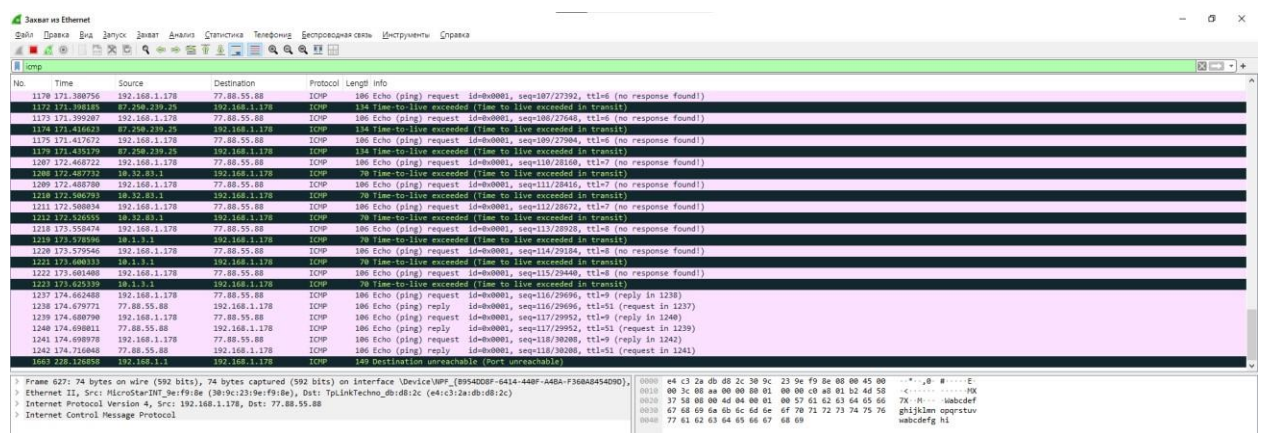


Рис. 10 – результаты отправки «traceroute»

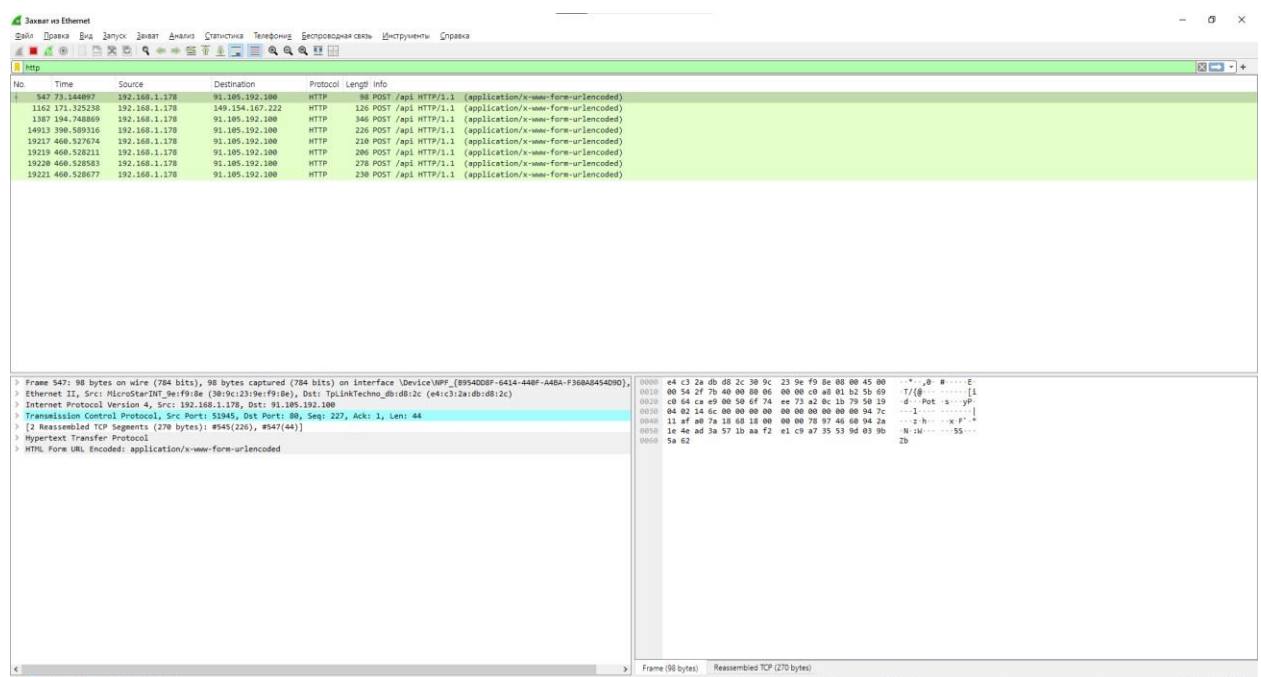


Рис. 11 - Результат с фильтрацией по протоколу «http»

Вывод:

В ходе лабораторной работы были изучены сетевые утилиты **Ping**, **Traceroute** и **Wireshark**, а также проанализированы результаты их работы при обращении к веб-сайту по протоколу HTTP.

1. **Утилита Ping** ◦ Позволяет проверить доступность узла в сети и измерить время прохождения пакетов (RTT — Round Trip Time). ◦ В ходе тестирования подтверждена корректность работы DNS, так как доменное имя успешно преобразовалось в IP-адрес. ◦ Наблюдаемые значения RTT и отсутствие потерь пакетов свидетельствуют о стабильном соединении с сервером. ◦ Высокий RTT или потеря пакетов могли бы указывать на перегрузку сети, проблемы с маршрутизацией или недоступность ресурса.
2. **Утилита Traceroute** ◦ Определила маршрут следования пакетов от источника до целевого узла, выявив промежуточные маршрутизаторы (хосты). ◦ Анализ результатов позволил выявить потенциальные узкие места: узлы с высоким временем отклика или отсутствием ответа (обозначены *), что может быть связано с фильтрацией ICMP-пакетов. ◦ Разница в количестве хопов для разных запусков (например, при использовании UDP/ICMP) объясняется особенностями маршрутизации и настройкой сетевого оборудования.
3. **Wireshark** ◦ Анализатор трафика подтвердил, что HTTP-запросы передаются в открытом виде (без шифрования), что делает их уязвимыми для перехвата. ◦ Зафиксирован процесс установления TCP-соединения (трехэтапное рукопожатие), передача HTTP GET-запроса и получение ответа с кодом статуса (например, 200 OK). ◦ В заголовках HTTP-пакетов обнаружены метаданные: User-Agent, Cookie, Referer, что демонстрирует механизмы идентификации клиента и управления сессией.
4. **Обращение к сайту по HTTP**
 - Убедились, что HTTP работает поверх TCP, обеспечивая надежную доставку данных. ◦ DNS-запросы предшествуют установлению соединения, подтверждая зависимость веб-сервисов от системы доменных имен.

- Отсутствие шифрования в HTTP подчеркивает необходимость использования HTTPS для защиты конфиденциальности данных.

Итоги

Комбинация утилит **Ping**, **Traceroute** и **Wireshark** позволяет комплексно диагностировать сетевые проблемы: от проверки доступности узла до анализа содержимого трафика. Понимание работы протокола HTTP и его уязвимостей важно для обеспечения безопасности и оптимизации веб-приложений. Полученные навыки полезны для администрирования сетей, разработки и отладки клиент-серверных систем.